TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

KIẾN TRÚC MÁY TÍNH

LAB 01

LÀM QUEN MARS: XỬ LÝ I/O và CẤU TRÚC ĐIỀU KIỆN

Họ và tên	Lê Thuận Phát	MSSV	25521360
Lớp	CE119.R11.VMTN.1	Ngày thực hành	6/10/2026
Tự chấm	9.5/10	Link git	https://github.com/phat-netizen/CE119.R11.VMTN.1_Lab_25521360.git

1. Tổng quan buổi thực hành

Sinh viên dùng trình mô phỏng **MARS 4.5** để biến kiến thức Chương 2 (kiến trúc bộ lệnh MIPS: toán hạng, số có dấu/không dấu, biểu diễn lệnh, lệnh điều kiện và nhảy) thành các thí nghiệm dự đoán – lập trình – quan sát – giải thích. Tình huống xuyên suốt: một chương trình tính tiền điện sinh hoạt theo biểu giá bậc thang, kèm một đoạn giai điệu ngắn phát bằng syscall MIDI để báo hoàn tất.

Sau buổi thực hành, Sinh viên có thể:

- Soạn, Assemble, chạy từng bước và gỡ lỗi chương trình MIPS32 trên MARS; dùng syscall nhập/xuất, thời gian hệ thống và MIDI.
- Phân biệt dòng lệnh nguồn, giả lệnh và lệnh cơ bản; đọc cột Code/Basic của Text Segment và tự giải mã các trường của lệnh dạng R, dạng I và độ lệch của lệnh nhánh.
- Hiện thực cấu trúc điều kiện và vòng lặp bằng `slt`/`sltu`/`beq`/`bne`; truy cập mảng trong bộ nhớ bằng địa chỉ cơ sở và chỉ số.
- Phân biệt so sánh có dấu và không dấu khi kiểm tra miền giá trị của dữ liệu nhập.
- Dùng Instruction Counter để đo số lệnh thực thi, xây dựng mô hình chi phí theo dữ liệu vào và đánh giá một thay đổi thiết kế bằng số liệu đo.

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen MARS, kiểm tra môi trường	15 phút	
TN 1	Từ lệnh nguồn đến mã máy: giả lệnh, Text Segment, Instruction Counter	20 phút	1,5
TN 2	Tính tiền điện bậc thang: nhập có kiểm tra, biểu giá lưu trong bộ nhớ	70 phút	4,0
TN 3	Phát giai điệu bằng syscall MIDI và đo thời gian chạy	25 phút	2,0
TN 4	Đo số lệnh và giải mã lệnh trên chương trình của mình	30 phút	2,0
Kết	Tổng kết, phản tư, nộp bài	20 phút	0,5
Tổng		**180 phút**	**10,0/10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- **Tham số cá nhân.** `a` là chữ số cuối của MSSV, `b` là chữ số liền trước. Mọi số liệu riêng của bài (đơn giá, nhịp độ, nhạc cụ, giá trị kiểm thử) được tính từ `a` và `b`; điền vào Bảng 1 trước khi bắt đầu.
- **Quy ước mã nguồn.** Mỗi chương trình có `.data`, `.text`, nhãn `main` và kết thúc bằng syscall 10. Chuỗi in ra và chú thích dùng ASCII không dấu. Ghi chú vai trò thanh ghi theo quy ước: `$t*` giá trị tạm, `$s*` giá trị cần giữ qua syscall, `$v0` số hiệu syscall, `$a0`–`$a3` đối số.
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi chạy trên MARS; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.

PHẦN 0 — Làm quen MARS (15 phút)

Giảng viên hướng dẫn nhanh giao diện MARS. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng dưới đây (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☑	Tạo file mới, lưu với tên `Lab01_<MSSV>_dien.asm`	Mỗi chương trình nộp là một file riêng; xem quy ước ở cuối tài liệu.
☑	Assemble (Run ▸ Assemble, F3), chạy (Run ▸ Go, F5), chạy từng lệnh (F7), lùi một lệnh (F8), đặt lại (F12)	Sau khi sửa mã phải Assemble lại. Lỗi Assemble hiện ở tab Mars Messages.
☑	Kiểm tra Settings: bỏ chọn **Delayed branching**; chọn **Permit extended (pseudo) instructions and formats**	Hai thiết lập này ảnh hưởng trực tiếp đến mã máy và số lệnh đếm được.
☑	Đọc các vùng của tab Execute: Text Segment (Address, Code, Basic, Source), Data Segment, Registers	Tích **Labels** ở dưới Data Segment để xem địa chỉ của nhãn dữ liệu.
☑	Đặt breakpoint (ô Bkpt) và chạy từng bước để quan sát thanh ghi đổi màu	Thanh ghi vừa bị ghi sẽ được tô sáng.
☑	Mở Tools ▸ Instruction Counter và bấm **Connect to MIPS** trước khi chạy	Công cụ chỉ đếm khi đã kết nối; bấm **Reset** để đếm lại từ 0.
☑	Tra bảng syscall trong Help ▸ Help (thẻ MIPS, mục Syscalls)	Bảng tóm tắt các syscall dùng trong buổi này nằm ngay dưới.
☑	Chụp màn hình cửa sổ MARS và dán vào khung ảnh của file này	Chụp cả vùng Run I/O lẫn vùng mã đang xét.

Các syscall và lệnh dùng trong buổi này

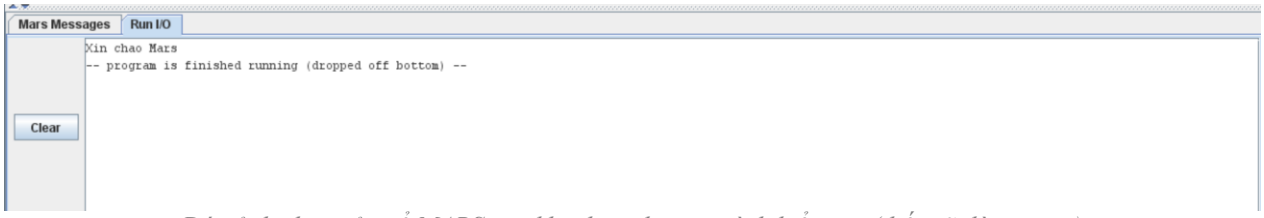
Đặt số hiệu dịch vụ vào `$v0`, đối số vào `$a0`–`$a3`, rồi thực hiện `syscall`.

$v0	Dịch vụ	Đối số	Kết quả
1	In số nguyên	`$a0` = giá trị	
4	In chuỗi kết thúc bằng 0	`$a0` = địa chỉ chuỗi	
5	Đọc số nguyên có dấu		`$v0` = giá trị đọc được
10	Kết thúc chương trình		
30	Lấy thời gian hệ thống (ms)		`$a0` = 32 bit thấp, `$a1` = 32 bit cao
31	Phát một nốt MIDI, trả về ngay	`$a0` cao độ (0–127), `$a1` độ dài (ms), `$a2` nhạc cụ (0–127), `$a3` âm lượng (0–127)	
32	Chờ (ngủ)	`$a0` = số ms	
33	Phát một nốt MIDI, chờ đến khi nốt kết thúc	như syscall 31	

Lệnh `mul rd, rs, rt` nhân hai số và ghi 32 bit thấp của tích vào `rd` (cơ chế bên trong sẽ học ở Chương 3, ở đây dùng như một lệnh có sẵn).

Mục	Ghi nhận
Phiên bản MARS đang dùng (Help ▸ About)	MARS 4.5
Máy có phát được âm thanh qua MIDI không? (Có / Không)	Có

Bước 0 – Kiểm tra nhanh. Tạo chương trình in dòng chữ `Xin chao MARS` rồi kết thúc (chỉ dùng syscall 4 và 10). Assemble, chạy và dán ảnh chụp vùng Run I/O vào khung dưới. File này chỉ để làm quen, không nộp.



Dán ảnh chụp cửa sổ MARS sau khi chạy chương trình kiểm tra (thấy rõ dòng in ra)

Tham số cá nhân

Điền `a`, `b` rồi tính các giá trị còn lại. Các giá trị này dùng cho TN 2 và TN 3.

Bảng 1. Tham số cá nhân

Đại lượng	Cách tính	Giá trị của Sinh viên
a	Chữ số cuối của MSSV	0
b	Chữ số liền trước chữ số cuối của MSSV	6
Đơn giá bậc 1 (đồng/kWh)	1600 + 20*a	1600
Số nửa cung dịch giọng	a	0
Độ dài một đơn vị nhịp (ms)	150 + 10*b	210
Nhạc cụ (số hiệu chương trình MIDI)	Phần dư của b khi chia 4 là 0, 1, 2, 3 tương ứng 0, 24, 40, 73	40
Số kWh kiểm thử cá nhân x_b	100 + 25*b	250

THÍ NGHIỆM 1 — Từ lệnh nguồn đến mã máy

Thời lượng gợi ý: 20 phút · File: tn1.asm (không nộp) · Điểm: 1,5

Tình huống

Trước khi viết chương trình lớn, kỹ sư cần biết một dòng mã nguồn thực sự trở thành bao nhiêu lệnh máy: số lệnh này quyết định kích thước chương trình và (cùng với số lệnh thực thi) thời gian chạy. Thí nghiệm này khảo sát một chương trình nhỏ để làm rõ ba con số: số dòng nguồn, số lệnh cơ bản trong bộ nhớ và số lệnh thực sự được thực thi.

Chương trình khảo sát tn1.asm (gõ lại nguyên văn, chưa Assemble)

```
        .data
msg:    .asciiz "Ket qua: "
        .text
main:   li   $t0, 25
        li   $t1, 70000
        add  $t2, $t0, $t1
        blt  $t0, $t1, small
        li   $t2, 0
small:  la   $a0, msg
        li   $v0, 4
        syscall
        move $a0, $t2
        li   $v0, 1
        syscall
        li   $v0, 10
        syscall
```

Bước 1.1 – Dự đoán trước khi Assemble. Chưa chạy MARS. Đọc chương trình và điền cột Dự đoán của Bảng 2 và Bảng 3.

DỰ ĐOÁN Mỗi dòng nguồn ở Bảng 2 sinh ra bao nhiêu lệnh cơ bản? Hai lệnh trong Bảng 3 là bao nhiêu dòng nguồn, lệnh cơ bản (trong bộ nhớ) và lệnh được thực thi? Chương trình in ra gì?

Bước 1.2 – Assemble và quan sát. Assemble, đọc Text Segment, điền cột Quan sát. Mở Instruction Counter (Connect to MIPS), chạy chương trình và đọc tổng số lệnh.

QUAN SÁT Cột Basic cho biết các lệnh cơ bản thực sự nằm trong bộ nhớ; cột Source là dòng nguồn tương ứng.

Bảng 2. Số lệnh cơ bản sinh ra từ mỗi dòng nguồn

Dòng nguồn	Dự đoán (số lệnh)	Quan sát (số lệnh)	Các lệnh cơ bản quan sát được (cột Basic)
`li $t0, 25`	1	1	addiu
`li $t1, 70000`	1	2	lui ; ori
`blt $t0, $t1, small`	8	2	Slt ; bne
`la $a0, msg`	2	2	Lui ; ori
`move $a0, $t2`	2	1	addu

Bảng 3. Số dòng lệnh nguồn, lệnh cơ bản, lệnh được thực thi của chương trình khảo sát

Đại lượng	Dự đoán	Quan sát	Nguồn đọc kết quả
Số dòng lệnh nguồn trong `.text`	13	13	Đếm trong Source
Số lệnh cơ bản trong bộ nhớ (tĩnh)	1	16	Số hàng của Text Segment
Số lệnh được thực thi (động)	3	15	Instruction Counter, mục Total
Chuỗi in ra Run I/O	Ket qua: 70025	Ket qua: 70025	Cửa sổ Run I/O

Câu 1.1 (Hiểu). Vì sao `li $t0, 25` chỉ cần một lệnh cơ bản còn `li $t1, 70000` cần hai? Giá trị 70000 không vừa trường nào của lệnh dạng I? Thanh ghi `$at` (số 1) xuất hiện ở đâu trong Text Segment và đóng vai trò gì?

Trả lời: +Dòng lệnh đầu chỉ nạp 25 nên chỉ cần 1 lệnh, còn lệnh sau nạp 70000 (lớn hơn lớn hơn 65535) nên cần tới 2.
+Giá trị 70000 không vừa trường imm của lệnh dạng I (dạng I chỉ có 16 bits choi mm)
+$at xuất hiện ở địa chỉ 0x00400004, đóng vai trò lưu trữ giá trị trung gian giữa các lệnh giả

Câu 1.2 (Phân tích). Giải thích vì sao ba con số ở Bảng 3 khác nhau. Trong hai lệnh `blt` và `li $t2, 0`, lệnh nào góp phần làm số lệnh thực thi nhỏ hơn số lệnh trong bộ nhớ và vì sao?

Trả lời:+Lệnh nguồn là lệnh mình ghi nên có 13 lệnh. Tuy nhiên để thực thi được thì cần them 3 lệnh giả mỗi cái cần 2 lệnh cơ bản nên tổng lệnh trong bộ nhớ là 16 , trong lúc chạy có 1 lệnh bị bỏ qua do branch nên có 15 lệnh
+Lệnh blt giúp số lệnh thực thi nhỏ hơn, vì nó branch tới small mà bỏ qua lệnh li $t2, 0

Bước 1.3 – Giải mã bằng tay. Đọc mã hex của lệnh `add $t2, $t0, $t1` ở cột Code **trước**, tách các trường rồi so với cột Basic. Làm tương tự với lệnh nhánh do `blt` sinh ra.

Bảng 4. Giải mã lệnh dạng R: `add $t2, $t0, $t1`

Địa chỉ	Mã hex (Code)	opcode	rs	rt	rd	shamt	funct
0x0040000c	0x01095020	000000	01000	01001	01010	00000	100000

Bảng 5. Giải mã lệnh nhánh sinh ra từ `blt` (lệnh thứ hai của cặp)

Địa chỉ	Mã hex (Code)	opcode	rs	rt	immediate (thập phân, có dấu)	Địa chỉ nhãn `small`
0x00400014	0x14200001	000101	00001	00000	+1	0x0040001c

Câu 1.3 (Áp dụng). Viết phép tính cho thấy trường immediate của lệnh nhánh ở Bảng 5 được suy ra từ địa chỉ của nó và địa chỉ nhãn `small` (nêu rõ vì sao có số hạng 4 và phép chia 4). Nếu chèn thêm 3 lệnh cơ bản vào giữa `blt` và `small`, trường immediate đổi thành bao nhiêu?

Trả lời:+Địa chỉ đích= (PC+4) +imm<<2 => imm=(địa chỉ đích – PC -4)>>2=(0x0040001c – 0x00400018) >>2=1
+Cộng 4 có là do khi lệnh nhánh chạy thì PC đã tang sẵn lên lệnh kế tiếp → nên độ lệch tính từ lệnh sau lệnh nhánh.
+Chia 4(dịch phải 2 bit): mỗi lệnh dài 4 byte, nên imm đếm theo số lệnh mà không theo byte →Phần cứng tự nhân 4 lại khi tính address →địa chỉ lệnh luôn chia hết cho 4 (2 bit cuối là 00), nên lưu 2 bit đó là lãng phí. Bỏ chúng đi thì 16 bit imm với được xa gấp 4 lần
+Nếu chèn thêm 3 lệnh cơ bản vào giữa thì imm sẽ bằng 4

Bước 1.4 – Dự đoán – quan sát lần hai. Trong tn1.asm thay `blt` bằng `bgt` (giữ nguyên nhãn và các dòng còn lại).

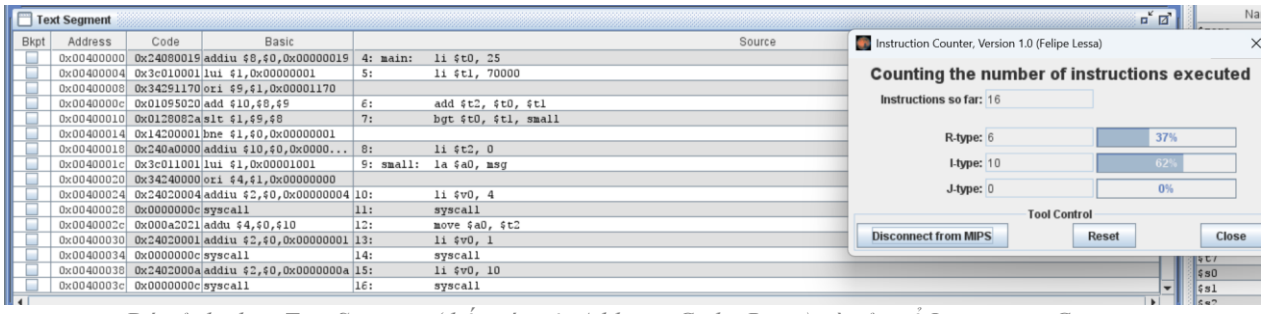
DỰ ĐOÁN Số lệnh thực thi bây giờ là bao nhiêu và chuỗi in ra là gì? Ghi dự đoán vào Bảng 6, Assemble, đặt lại Instruction Counter, chạy và ghi quan sát.

Bảng 6. Thay `blt` bằng `bgt`

Đại lượng	Dự đoán	Quan sát
Số lệnh được thực thi	16	16
Chuỗi in ra	Ket qua: 0	Ket qua: 0

Câu 1.4 (Phân tích). Ba lệnh `syscall` trong chương trình có cùng mã máy hay khác nhau (ghi mã hex quan sát được)? Nếu giống nhau thì máy phân biệt dịch vụ cần thực hiện bằng cách nào?

Trả lời: +Ba lệnh syscall trong chương trình có cùng mã mày là 0x0000000c.
+Máy phân biết dịch vụ cần thực hiện bằng số được lưu trong các thanh ghi như $v0.



Dán ảnh chụp Text Segment (thấy các cột Address, Code, Basic) và cửa sổ Instruction Counter

THÍ NGHIỆM 2 — Tính tiền điện bậc thang

Thời lượng gợi ý: 70 phút · File nộp: Lab01_<MSSV>_dien.asm · Điểm: 4,0

Tình huống

Một công ty điện lực cần chương trình tính tiền điện sinh hoạt của một hộ trong một tháng. Điện năng tiêu thụ được chia thành sáu bậc; **mỗi kWh được tính theo đơn giá của bậc chứa nó**, và tổng tiền là tổng của các phần ở từng bậc. Biểu giá dưới đây là biểu giá giả lập dùng cho bài thực hành. Chương trình chạy trên một vi xử lý nhúng nhỏ nên bảng giá phải nằm trong bộ nhớ dữ liệu để thay đổi khi biểu giá đổi mà không phải sửa phần lệnh.

Bảng 7. Biểu giá giả lập (đồng/kWh, số nguyên)

Bậc	Từ (kWh)	Đến (kWh)	Đơn giá (đồng/kWh)
1	0	50	1600 + 20*a
2	51	100	1660 + 20*a
3	101	200	1950 + 20*a
4	201	300	2500 + 20*a
5	301	400	2850 + 20*a
6	401	9999	3000 + 20*a

Đặc tả chương trình

- **Nhập.** In lời nhắc `Nhap so kWh (0..9999): ` rồi đọc một số nguyên có dấu bằng syscall 5.
- **Kiểm tra.** Giá trị hợp lệ khi $0 \le x \le 9999$. Nếu không hợp lệ, in `Gia tri khong hop le, nhap lai.` rồi nhắc nhập lại cho đến khi hợp lệ. Việc kiểm tra miền giá trị chỉ được dùng **một** lệnh so sánh và **một** lệnh nhánh.
- **Dữ liệu.** Ngưỡng các bậc và đơn giá lưu trong `.data` dưới dạng mảng `.word`. Phần tính toán duyệt các mảng này bằng vòng lặp; đổi giá trị trong mảng thì không phải sửa phần lệnh. Không viết riêng mã cho từng bậc.
- **Xuất.** In đúng khuôn `Tien dien: <so tien> dong` rồi xuống dòng, sau đó kết thúc chương trình.
- **Giao diện thanh ghi.** Ghi chú ở đầu file vai trò của từng thanh ghi `$s*` đã dùng.

Bước 2.1 – Dự đoán bằng tay. Chưa viết mã. Điền đơn giá 6 bậc của Sinh viên (từ Bảng 1) vào Bảng 8, rồi tính tay tiền điện cho các giá trị x ở Bảng 10 (có thể dùng máy tính bỏ túi).

DỰ ĐOÁN Ghi tiền điện dự kiến cho từng x. Chú ý các giá trị nằm đúng trên hoặc sát ranh giới bậc.

Bảng 8. Đơn giá 6 bậc theo tham số a của Sinh viên

Bậc	1	2	3	4	5	6
Đơn giá	1600	1660	1950	2500	2850	3000

Bước 2.2 – Thiết kế trước khi viết mã. Mô tả thuật toán bằng lời hoặc mã giả (không cần đúng cú pháp) và lập bảng phân công thanh ghi.

THIẾT KẾ Ghi rõ điều kiện kết thúc vòng lặp, cách xác định phần điện năng thuộc mỗi bậc và vì sao chương trình không cần biết trước số bậc đã dùng.

Thuật toán: +Khởi tạo mảng gia chứa các đơn giá và mảng nguong chứa ngưỡng các bậc
+ Nhập kWh vào nếu <0 hoặc >9999 thì nhập lại
+ logic mỗi vòng lặp:
-cập nhật ngưỡng và giá tiền tại bậc đó
-Nếu số kWh <= ngưỡng hiện ra thì nhảy sang label bậc cuối và sẽ đc tính ra số tiền tại bậc cuối
-Không nhảy xuống bậc cuối thì tiếp tục: Lấy ngưỡng – cận dưới = số kWh của bậc hiện tại, tính được số tiền tại bậc đó rồi cộng vào tổng tiền
-Cập nhật address của ngưỡng và tiền rồi nhảy lại lap
-Vòng lặp luôn dừng vì ngưỡng cuối là 9999, lớn hơn hoặc bằng mọi x hợp lệ, nên không cần biết trước số bậc.

Bảng 9. Phân công thanh ghi

Thanh ghi	Vai trò trong chương trình
$s0	con tro vao mang nguong (tang 4 moi vong)
$s1	x, so kWh nhap vao
$s2	Số tiền phải trả
$s4	Cận dưới (lưu ngưỡng của bậc hiện tại -1
$t0	GIá trị trung gian: ngưỡng của bậc đang xét
$t1	Giá trị trung gian: Chứa đơn giá của bậc đang xét
$t2	Giá trị trung gian: số kWh của bậc hiện tại, giá tiền của bậc hiện tại

Bước 2.3 – Lập trình. Viết chương trình theo đặc tả trong file `Lab01_<MSSV>_dien.asm`. Assemble, sửa lỗi, chạy từng bước ít nhất một lần để theo dõi vòng lặp qua các bậc.

LẬP TRÌNH Khi gỡ lỗi, đặt breakpoint ở đầu thân vòng lặp và theo dõi các thanh ghi tổng tiền, cận dưới và con trỏ vào mảng.

Bước 2.4 – Kiểm thử và so sánh với dự đoán. Chạy chương trình lần lượt với các giá trị ở Bảng 10 và Bảng 11.

QUAN SÁT Ghi kết quả chương trình in ra và đối chiếu với dự đoán. Nếu lệch, tìm xem dự đoán tay hay chương trình sai, ghi kết luận vào cột cuối.

Bảng 10. Tiền điện: dự đoán và quan sát

x (kWh)	Dự đoán (đồng)	Quan sát (đồng)	Khớp? Nếu lệch, nguyên nhân
30	48000	48000	Khớp
50	80000	80000	Khớp
51	81 660	81 660	Khớp
120	202000	202000	Khớp
250	483000	483000	Khớp
401	896000	896000	Khớp
x_b = 250	483000	483000	Khớp

Bảng 11. Dữ liệu nhập không hợp lệ và giá trị biên của miền nhập

Dữ liệu nhập	Dự đoán hành vi	Quan sát
0	Nhận giá trị	Nhận giá trị
9999	Nhận giá trị	Nhận giá trị
10000	Yêu cầu nhập lại	Yêu cầu nhập lại
−1	Yêu cầu nhập lại	Yêu cầu nhập lại

Câu 2.1 (Phân tích). Chỉ rõ lệnh so sánh và lệnh nhánh dùng để kiểm tra miền [0, 9999] trong chương trình của Sinh viên. Giải thích vì sao **một** phép so sánh bắt được cả x < 0 lẫn x ≥ 10000.

Trả lời:
+Nhập lại nếu $s1>9999 hoặc $s1<0 : sltiu $t0, $s1, 10000
+Giải thích: Vì lệnh sltiu $t0, $s1, 10000 sử dụng so sánh 2 số không dấu thế nên nếu số nhập âm thì MSB=1 sẽ luôn lớn hơn 9999. Khi nhập n (0<=n<=9999) thì MSB =0 và giá trị không dấu thỏa <=9999 nên $t0 sẽ được set=1.
+Lệnh nhánh sẽ nhảy vào hợp lệ khi có $t0=1, còn không thì sẽ in ra string báo lỗi và quay về nhap lại lần nữa.

Bước 2.5 – Thí nghiệm có dấu và không dấu. Đổi lệnh so sánh ở câu 2.1 sang phiên bản so sánh **ngược** kiểu (có dấu thành không dấu hoặc ngược lại), Assemble lại, nhập x = −1.

DỰ ĐOÁN Chương trình sẽ làm gì? Ghi dự đoán trước, rồi chạy và ghi quan sát. Sau khi xong, khôi phục lệnh đúng.

Dự đoán	Quan sát
Nhận giá trị	Nhận giá trị và tổng tiền = -1600

Câu 2.2 (Phân tích). Giải thích kết quả quan sát bằng cách xét biểu diễn bù hai của −1 và cách hai lệnh so sánh diễn giải cùng một chuỗi bit.

Trả lời:
-1=0xFFFFFFFF
+slti $t0, $s1,10000,Nhap thì so sánh: -1<10000 ĐÚNG nên $t0 =1
+sltiu $t0,$s1,10000,Nhap thì so sánh: 2^32-1<10000 SAI nên $t0=0
➔ Vì −1 không bị bắt nên chương trình chạy tiếp

Bước 2.6 – Quan sát bộ nhớ. Mở Tools → tích **Labels**, quan sát cửa sổ Labels và tìm địa chỉ nhãn của mảng đơn giá trong Data Segment. Đọc phần tử thứ 3 (chỉ số 2) ở dạng hex và thập phân.

QUAN SÁT Đối chiếu địa chỉ phần tử với địa chỉ nhãn và với lệnh nạp dữ liệu tương ứng trong chương trình.

Bảng 12. Mảng đơn giá trong Data Segment

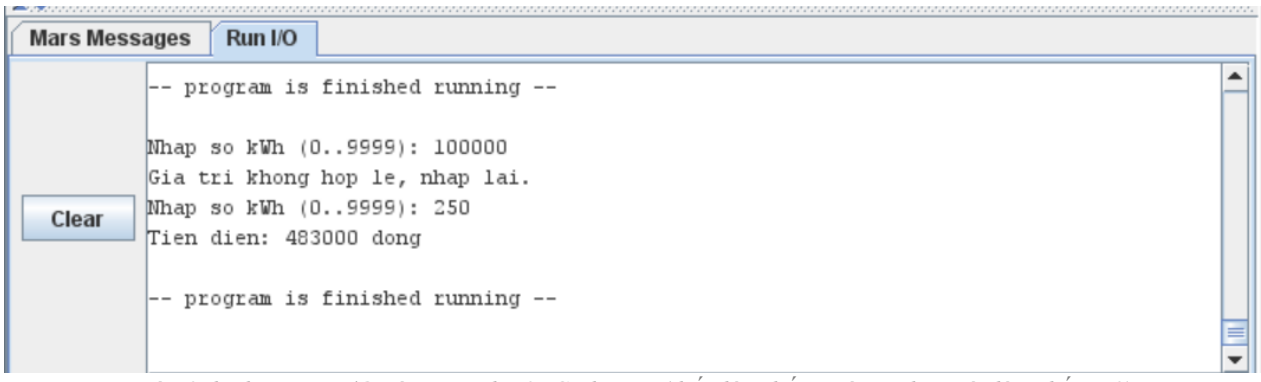
Địa chỉ nhãn mảng	Địa chỉ phần tử thứ 3	Giá trị (hex)	Giá trị (thập phân)
gia= 0x10010018	0x10010020	0x0000079E	1950
nguong= 0x10010000	0x10010008	0x000000C8	200

Câu 2.3 (Áp dụng). Tính tổng tiền lớn nhất mà chương trình có thể in ra với miền nhập hợp lệ và cho biết phép cộng có thể gây tràn có dấu 32 bit hay không. Nếu miền nhập mở rộng đến N kWh thì N lớn nhất là bao nhiêu để tổng tiền vẫn không tràn? Nêu cách tính.

Trả lời:
+Tổng tiền lớn nhất là khi nhập 9999: 29690000 đồng =>chưa xảy ra tràn số
+Nếu không thay đổi ngưỡng và số tiền các ngưỡng thì N có thể lên đến 715930 kWh
+Cách tính N = [(2^{31} − 1 − 29690000) / 3000] + 9999 = 715930 (làm tròn xuống), vì tràn có dấu xảy ra khi vượt 2^{31} − 1 = 2147483647

Câu 2.4 (Đánh giá). So sánh thiết kế duyệt mảng của Sinh viên với cách viết sáu nhánh `if` riêng cho từng bậc: nêu ít nhất hai tiêu chí (ví dụ kích thước mã, khi biểu giá đổi, khi thêm bậc) và kết luận phương án nào hợp lý hơn cho thiết bị nhúng trong tình huống này.

Trả lời:
+Duyệt mảng: Khi thêm bậc hoặc đổi giá chỉ cần thêm, thay đổi trong mảng (logic lap giữ nguyên), kích thước mã ngắn gọn không dài dòng
+Nhành if: Khi đổi giá thì chỉ cần them trong mảng, nhưng khi thêm bậc thì cần viết thêm số lượng khối lệnh if tương ứng, kích thước mã dài dòng.
➔Phương án duyệt mảng phù hợp hơn, dễ dàng bổ sung, thay đổi khi cần thiết hơn.



Dán ảnh chụp Run I/O với x = x_b của Sinh viên (thấy lời nhắc, giá trị nhập và dòng kết quả)

THÍ NGHIỆM 3 — Phát giai điệu bằng syscall MIDI

Thời lượng gợi ý: 25 phút · File nộp: Lab01_<MSSV>_nhac.asm · Điểm: 2,0

Tình huống

Thiết bị cần phát một giai điệu ngắn để báo hoàn tất. Giai điệu lưu dưới dạng hai mảng (cao độ và độ dài) trong bộ nhớ dữ liệu, chương trình duyệt mảng và gọi syscall MIDI cho từng nốt. Giai điệu là chủ đề *Ode to Joy* của L. van Beethoven (thuộc phạm vi công cộng); cao độ theo số hiệu nốt MIDI (60 là C4).

Bảng 13. Dữ liệu giai điệu (độ dài tính theo đơn vị nhịp, 1 đơn vị = giá trị `150 + 10*b` ms ở Bảng 1)

Nốt	Tên	Cao độ	Độ dài	Nốt	Tên	Cao độ	Độ dài
1	E4	64	2	9	C4	60	2
2	E4	64	2	10	C4	60	2
3	F4	65	2	11	D4	62	2
4	G4	67	2	12	E4	64	2
5	G4	67	2	13	E4	64	3
6	F4	65	2	14	D4	62	1
7	E4	64	2	15	D4	62	4
8	D4	62	2				

Đặc tả chương trình

- Hai mảng `.word` trong `.data` chứa cao độ và độ dài (đơn vị nhịp) của 15 nốt; số nốt cũng là một dữ liệu trong `.data`.
- Với mỗi nốt: cao độ thực phát bằng cao độ trong mảng cộng `a`; thời lượng (ms) bằng độ dài nhân với `150 + 10*b`; nhạc cụ và âm lượng 100 theo tham số cá nhân. In ra một dòng `<cao do> <ms>` rồi phát nốt bằng syscall 33.
- Đo thời gian chạy của cả vòng phát bằng syscall 30 (hiệu của hai lần đọc `$a0`). Cuối chương trình in `Tong du kien (ms): <so>` (chương trình tự cộng dồn thời lượng các nốt) và `Thoi gian do duoc (ms): <so>`.

Bước 3.1 – Dự đoán. Dùng dữ liệu của Sinh viên trước khi viết mã.

DỰ ĐOÁN Tổng thời lượng dự kiến của 15 nốt là bao nhiêu ms? Nếu thay syscall 33 bằng syscall 31 (giữ nguyên tham số), điều gì xảy ra với âm thanh và với thời gian đo được?

Tổng dự kiến (ms)	Dự đoán khi thay 33 bằng 31
6720	1-3 ms

Bước 3.2 – Lập trình và quan sát. Viết `Lab01_<MSSV>_nhac.asm` theo đặc tả, chạy và ghi số liệu vào Bảng 14. Nếu máy không phát được âm thanh, vẫn làm đủ các bước: ghi "Không nghe được" ở cột Nghe thấy và dựa vào các dòng in ra và thời gian đo được.

QUAN SÁT Nghe giai điệu có đúng nhịp không? Đối chiếu dòng in cuối với dự đoán ở Bước 3.1.

Bước 3.3 – So sánh ba cách phát. Thử lần lượt ba cấu hình và điền Bảng 14: (A) syscall 33; (B) syscall 31 thay cho 33; (C) syscall 31 rồi ngay sau đó syscall 32 với `$a0` bằng độ dài nốt (ms).

Bảng 14. Ba cấu hình phát nốt

Cấu hình	Nghe thấy	Thời gian đo được (ms)	Số dòng in ra	Nhận xét ngắn
A: 33	Nghe thấy	7509	17	Phát âm thanh rõ rang, nhưng thời gian đo được có chênh lệch mỗi lần đo (7509-7556, có 1 lần chạy 17000ms)
B: 31	Nghe thấy	1	17	Phát tất cả âm thanh cùng một lúc
C: 31 + 32	Nghe thấy	6727	17	Phát âm thanh rõ ràng

Câu 3.1 (Phân tích). Giải thích chênh lệch (nếu có) giữa tổng dự kiến và thời gian đo được ở cấu hình A. Liệt kê các nguồn gây chênh lệch mà Sinh viên nhận ra.

Trả lời:
+Có sự chệnh lệnh do: tổng dự kiến chỉ tính thời gian chạy của mỗi nốt, còn thời gian đo được là thời gian giữa 2 lần gọi syscall 30, trong đó bao gồm thời gian chạy các lệnh in, khởi tạo các thiết bị MIDI, đợi hệ điều hành xử lí.
+Các nguồn gây nhiễu là: Chạy lệnh, in ra màn, tốc độ xử lí của hệ điều hành, sai số đo

Câu 3.2 (Áp dụng). Nếu mảng cao độ khai báo bằng `.byte` thay vì `.word`, lệnh nạp phần tử và cách tính địa chỉ phần tử thứ i thay đổi thế nào? Chọn `lb` hay `lbu` cho trường hợp cao độ có thể lên đến 127 và cho trường hợp tổng quát 0–255? Vì sao?

Trả lời:
+Khai báo bằng byte thì mỗi phần tử chỉ cách nhau một byte thế nên truy xuất phần tử tiếp theo chỉ cần +1 (còn word thì cần +4). Ngoài ra cần thay đổi lệnh lw (load word) thành lb (load byte) thì mới đọc được kiểu dữ liệu byte
*+khi giá trị ≤ 127 thì **bit 7 = 0**, nên lb và lbu cho cùng kết quả.*
+Nên chọn lệnh lbu cho cao độ ,trường hợp tổng quát thì có thể bị diễn giải thành số âm cho lệnh lb vì lb có 7 bít số và 1 bit dấu nên lưu được từ (-128 →+127), còn lbu đọc dữ liệu 8 bit nên đọc đc từ 0 →255
VD: cao độ =200 thì qua lb sẽ thành -56, còn lbu sẽ vẫn là 200

Câu 3.3 (Đánh giá). Dựa trên Bảng 14, thiết bị vừa phải phát nhạc vừa phải đọc cảm biến thì nên dùng cấu hình nào? Nêu một rủi ro khi dùng cấu hình B mà không chờ.

Trả lời:
- ***Chọn C**: đúng nhịp nhất (6727 ms, chỉ lệch +7 ms so với dự kiến 6720 ms). Phần chờ do chương trình tự gọi syscall 32, nên có thể tận dụng khoảng chờ đó để đọc cảm biến. A lệch +789 ms và chờ nằm trong syscall 33, không can thiệp được. B hỏng giai điệu.*
- ***Rủi ro của B khi không chờ:** syscall 31 trả về ngay (khoảng 1 ms cho cả vòng lặp), 15 nốt phát gần như cùng lúc, nên chỉ nghe được một âm chồng lấn và mất nhịp.*

THÍ NGHIỆM 4 — Đo số lệnh và giải mã lệnh trên chương trình của mình

Thời lượng gợi ý: 30 phút · Dùng chương trình `Lab01_<MSSV>_dien.asm` đã hoàn thành · Điểm: 2,0

> **Tình huống**
> Trước khi triển khai lên vi xử lý nhúng có tài nguyên hạn chế, nhóm phát triển cần ước lượng chi phí của chương trình theo dữ liệu vào và hiểu rõ mã máy sinh ra. Sinh viên đo số lệnh thực thi của chương trình tính tiền điện, xây dựng mô hình chi phí đơn giản và giải mã một số lệnh của chính chương trình mình.

Bước 4.1 – Đo số lệnh thực thi. Mở Instruction Counter (Connect to MIPS). Với mỗi x ở Bảng 15, đặt lại bộ đếm, chạy, ghi tổng số lệnh thực thi. Đo x = 10 và x = 120 trước.

DỰ ĐOÁN Từ hai số đo đầu tiên, đề xuất công thức N(x) và điền cột Dự đoán cho các hàng còn lại; sau đó đo các hàng còn lại và so sánh.

Bảng 15. Số lệnh thực thi theo số kWh nhập

x (kWh)	Số bậc đã dùng hết	N dự đoán	N đo được	Lệch
10	0	—	35	—
120	2	—	57	—
50	0	35	35	0
250	3	68	68	0
401	5	90	90	0

Câu 4.1 (Phân tích). Viết công thức N(x) của chương trình Sinh viên và giải thích ý nghĩa từng hệ số bằng cách đếm số lệnh trong phần chuẩn bị, trong thân vòng lặp mỗi bậc và trong phần in kết quả. Vì sao N không đổi với mọi x thuộc cùng một bậc?

Trả lời:
+ N(x) = 35 + 11·k
+Số Lệnh phần chuẩn bị: 15 lệnh cơ bản gồm nhập, khởi tạo, biến
+Mỗi vòng lặp (mỗi bậc): 11 lệnh cơ bản
+Phần in kết quả : 13 lệnh
+Lệnh trong lần lặp cuối : 7 lệnh
+Cố định là: 15+13+7=35
*+N không đổi trong cùng một bậc vì vòng lặp chạy theo **số bậc**, không theo số kWh. x chỉ xuất hiện trong một phép nhân.*

Câu 4.2 (Đánh giá). Nếu biểu giá có 60 bậc và x nằm ở bậc cuối, N xấp xỉ bao nhiêu? Đề xuất **một** thay đổi nhằm giảm số lệnh mỗi vòng, thực hiện và đo lại ở x = 401; báo cáo N trước, N sau và phần trăm giảm. Nếu không giảm được, lập luận vì sao thân vòng đã tối thiểu.

*Bảng 15 và Câu 4.1 đo trên bản trước khi sửa; file nộp là bản đã sửa (N = 33 + 10k)

Trả lời:
*+ N = 35 + 11·59 = **684 lệnh***
+Mảng giá và ngưỡng liền nhau → không cần lưu 2 địa chỉ mảng mà chỉ cần một cái $s0= address ngưỡng →ngương =0($s0) và giá = 24($s0)
+N trước =90, N sau = 83 →giảm 7 lệnh (7,8%)

Bước 4.2 – Giải mã lệnh của chương trình mình. Trong Text Segment của chương trình tiền điện, chọn ba lệnh: một lệnh nhánh trong vòng lặp, một lệnh nạp từ mảng và một lệnh so sánh. Với mỗi lệnh, ghi địa chỉ, mã hex, tách các trường và **tự đọc ra lệnh trước khi nhìn cột Basic**.

QUAN SÁT So khớp kết quả giải mã với cột Basic. Nếu khác, tìm trường nào bị đọc sai.

Bảng 16. Giải mã ba lệnh của chương trình tiền điện

Địa chỉ	Mã hex	Dạng lệnh	Các trường (tên = giá trị thập phân)	Lệnh đọc ra
0x00400054	0x10200006	I	op = 4, rs = 1, rt = 0, imm = 6	`beq $1, $0, 6`
0x0040004C	0x8E090018	I	op = 35, rs = 16, rt = 9, imm = 24	`lw $9, 24($16)`
0x00400050	0x0111082A	R	op = 0, rs = 8, rt = 17, rd = 1, shamt = 0, funct = 42	`slt $1, $8, $17`

Câu 4.3 (Áp dụng). Với lệnh nhánh đã chọn, tính (địa chỉ đích − (địa chỉ lệnh + 4)) / 4 và so với trường immediate trong mã hex. Với trường immediate 16 bit, một lệnh nhánh đến được các địa chỉ trong khoảng bao nhiêu byte so với lệnh kế tiếp?

Trả lời:
+Lệnh beq tại 0x00400054 nhảy tới bậc cuối ở 0x00400070: (0x00400070 − (0x00400054 + 4)) / 4 = (0x70 − 0x58) / 4 = 24 / 4 = 6, trùng với trường imm = 6 trong mã 0x10200006.
+imm có 16 bit chứa từ -2^15 → 2^15-1 ⇔ (-32768 →32767) → nhân 4 ra số byte: (−131072 → +131068 byte)

Bảng 17. Giả lệnh trong chương trình tiền điện

Giả lệnh trong mã nguồn	Lệnh cơ bản sinh ra	Số lệnh
`la $s0, nguong`	`lui$1,0x1001` `ori $16, $1, 0x0000`	2
`ble $s1, $t0, bac_cuoi`	`slt$1,$8,$17` `beq $1, $0, 6`	2
`move $s4, $t0`	`addu $20, $0, $8`	1

Câu 4.4 (Đánh giá). Chương trình của Sinh viên có bao nhiêu dòng lệnh nguồn và bao nhiêu lệnh cơ bản trong bộ nhớ? Nếu một giả lệnh nạp địa chỉ nằm trong thân vòng lặp thì nó ảnh hưởng thế nào đến N(x)? Nêu cách bố trí chương trình để tránh chi phí này.

Trả lời:
+ 38 dòng lệnh nguồn.
+ 44 lệnh cơ bản trong bộ nhớ. (Lệch do có 6 lệnh giả, mỗi lệnh sinh ra 2 lệnh cơ bản)
*+Nếu thêm một giả lệnh mà giả lệnh đó được xử lí bằng 2 lệnh cơ bản thì N tăng thêm **2·(k + 1)** lệnh vd: các lệnh la*
+Cách bố trí cho lệnh giả ngoài vòng lặp vd: cho lệnh la ở ngoài, trong vòng lặp chỉ tang con trỏ

TỔNG KẾT VÀ NỘP BÀI (20 phút)

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File `Lab01_<MSSV>_dien.asm` Assemble không lỗi, có `.data`, `.text`, nhãn `main`, kết thúc bằng syscall 10 và chú thích vai trò thanh ghi.
☑	File `Lab01_<MSSV>_nhac.asm` Assemble không lỗi và in đúng khuôn các dòng yêu cầu.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp vào các khung nét đứt.
☑	Số liệu trong báo cáo khớp với chương trình nộp (đơn giá, tham số cá nhân và kết quả đo).
☑	Đặt tên file báo cáo: `Lab01_<MSSV>_BaoCao.docx`; nộp ba file theo hướng dẫn của giảng viên.

Tự chấm

Phần	Điểm tối đa	Tự chấm
TN 1	1,5	1.5
TN 2	4,0	4.0
TN 3	2,0	1.75
TN 4	2,0	1.75
Kết	0,5	0.5

Phản tư

Câu R1 (Đánh giá). Từ TN 1 và TN 4, Sinh viên rút ra điều gì về sự khác nhau giữa số dòng nguồn, số lệnh trong bộ nhớ và số lệnh thực thi? Nêu một tình huống cần quan tâm đến từng con số.

Trả lời:
+Dòng nguồn là số câu lệnh được ghi vào, không phân biết lệnh giả hay cơ bản
+Số lệnh trong bộ nhớ là số lệnh cơ bản được lưu trong text segment (một lệnh giả có thể cần xử lý bởi nhiều lệnh cơ bản)
+Số lệnh thực thi: là số lệnh được cpu xử lí (có thể phụ thuộc vào input đầu vào, các lệnh nhảnh, vòng lặp)
+Tình huống cần quan tâm: liên quan đến việc người đọc code và người bảo trì, bộ nhớ chương trình có giới hạn, hời gian chạy hoặc năng lượng, đặc biệt là trường hợp xấu nhất

Câu R2 (Phân tích). Từ TN 2, kiểu so sánh có dấu hay không dấu quyết định điều gì trong chương trình? Nêu một tình huống khác ngoài kiểm tra miền nhập mà chọn sai sẽ gây lỗi.

Trả lời:
+Kiểu so sánh quyết định cách hiểu cùng một dãy bit, từ đó quyết định nhánh nào được chọn. Ví dụ x = −1 (0xFFFFFFFF): slti hiểu là −1 < 10000 nên coi là hợp lệ (sai), còn sltiu hiểu là 4294967295 ≥ 10000 nên loại (đúng).
+Tình huống khác: vòng lặp đếm lùi i >= 0 mà dùng so sánh không dấu. Khi i = −1 thì giá trị không dấu là 4294967295, vẫn ≥ 0, nên vòng lặp không dừng và đọc ra ngoài mảng.

Câu R3 (Đánh giá). Nêu một lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Trả lời:
+Ghi đè thanh ghi nên giá trị cũ bị mất
+Phát hiện bằng cách chạy thử thì khác với đáp án tay
+Bài học rút ra là lập bảng ghi vai trò của thanh ghi, vừa viết code vừa kiểm tra đúng thanh ghi chưa.

Tài liệu tham khảo

- Slide bài giảng Chương 2 – Kiến trúc bộ lệnh (Khoa Kỹ thuật Máy tính, UIT).
- MARS 4.5, Help ▸ Help: các mục Instructions, Pseudo-instructions, Syscalls.